📋 DOCUMENTAÇÃO COMPLETA — SITE DE CONFIRMAÇÃO DE EVENTO

1. Objetivo do projeto

O projeto foi criado para ser um **site de confirmação de presença para eventos**, inicialmente desenvolvido para a festa de **15 anos da Yasmim**.

A ideia foi criar uma estrutura completa contendo:

Área pública

- Página inicial do evento;
- informações da aniversariante;
- contador regressivo;
- localização;
- galeria de fotos;
- confirmação de presença;
- confirmação individual dos convidados;
- página de confirmação final;
- lista de presentes;
- reserva de presentes;
- mensagens.

Área administrativa

- Login administrativo;
- Dashboard;
- cadastro de famílias;
- edição de famílias;
- exclusão de famílias;
- visualização dos convidados;
- controle das confirmações;
- cadastro de presentes;
- edição de presentes;
- exclusão de presentes;
- controle de presentes reservados;
- mensagens dos convidados;
- galeria de fotos;
- exclusão de fotos;
- recuperação/redefinição de senha.

A estrutura foi pensada para posteriormente ser **reutilizada para outros clientes**, trocando principalmente cores, textos, imagens, aniversariante, famílias, convidados e presentes.

---

2. Tecnologias utilizadas

O projeto ficou dividido em duas aplicações:

```
SITE-CONFIRMA
│
├── FRONTEND
│
└── BACKEND
```

Frontend

Foi utilizado:

- React
- TypeScript
- Vite
- React Router
- Tailwind CSS
- Lucide React
- Fetch API

Backend

Foi utilizado:

- Node.js
- Express
- TypeScript
- Prisma ORM
- PostgreSQL
- bcrypt
- Multer, para recebimento de arquivos/imagens
- Cloudinary, para armazenamento das imagens

Serviços externos

Foram utilizados:

Serviço	Função
GitHub	Armazenamento do código
Neon	Banco de dados PostgreSQL
Cloudinary	Armazenamento das fotos
Render	Deploy do backend
Vercel	Deploy do frontend

Repositório utilizado:

GitHub — Site-confirma

---

3. Primeiro desenvolvimento — Frontend

O frontend foi desenvolvido como uma aplicação React com Vite.

A responsabilidade dele é cuidar de **tudo que o usuário vê e utiliza no navegador**.

Por exemplo:

```
FRONTEND
   ↓
Páginas
   ↓
Componentes
   ↓
Formulários
   ↓
Chamadas para a API
   ↓
BACKEND
```

O frontend não acessa diretamente o banco de dados.

Ele conversa com o backend através de requisições HTTP.

---

4. Estrutura do FRONTEND

A estrutura principal foi organizada aproximadamente desta forma:

```
FRONTEND
│
├── src
│   │
│   ├── app
│   │   │
│   │   ├── admin
│   │   │   ├── _components
│   │   │   ├── modals
│   │   │   └── pages
│   │   │
│   │   ├── public
│   │   │   ├── _components
│   │   │   └── pages
│   │   │
│   │   └── shared
│   │
│   ├── config
│   │
│   ├── services
│   │
│   ├── styles
│   │
│   └── index.css
│
├── public
│
├── package.json
├── vite.config...
└── ...
```

---

5. `src/app/public`

Essa pasta contém a **parte pública do site**.

É aquilo que o convidado acessa.

public/pages

Contém as páginas principais.

Entre elas:

```
Inicio.tsx
Presentes.tsx
ConfirmacaoFinal.tsx
```

Inicio.tsx

É a página inicial do evento.

Nela ficam as principais informações, como:

- nome da aniversariante;
- informações do evento;
- data;
- horário;
- localização;
- elementos visuais;
- acesso às outras áreas.

Presentes.tsx

Responsável pela lista de presentes.

Ela:

1. busca os presentes no backend;
2. mostra os presentes;
3. permite reservar;
4. atualiza a situação do presente.

Exemplo da lógica utilizada:

```
const dados = await listarPresentes();
setPresentes(dados);
```

Para reservar:

```
await atualizarPresente({
  ...presenteSelecionado,
  reservado: true,
  reservador: nome,
});
```

ConfirmacaoFinal.tsx

É a página exibida depois da confirmação.

Ela mostra a confirmação final ao convidado.

---

6. `src/app/public/_components`

Aqui ficam os componentes utilizados pela parte pública.

Entre eles:

```
BarraNavegação.tsx
CardPresente.tsx
CarrosselFotos.tsx
ContadorRegressivo.tsx
```

`BarraNavegação.tsx`

Menu de navegação do site.

`CardPresente.tsx`

Mostra cada presente disponível.

Quando o presente já foi reservado, o botão muda para algo como:

```
Presente Reservado
```

`CarrosselFotos.tsx`

Exibe as fotos da galeria.

`ContadorRegressivo.tsx`

Mostra a contagem regressiva para o evento.

---

7. `src/app/admin`

Essa é a área administrativa.

O administrador consegue controlar o conteúdo do evento.

Estrutura:

```
admin
│
├── _components
├── modals
└── pages
```

---

8. `admin/pages`

Aqui ficam as páginas administrativas.

Principais arquivos:

```
AdminLogin.tsx
AdminCadastro.tsx
AdminConvidados.tsx
AdminPresentes.tsx
AdminMensagens.tsx
AdminGaleria.tsx
```

`AdminLogin.tsx`

Tela de acesso administrativo.

O código atual utiliza:

```
const admin = await loginAdmin(email, senha);

salvarAdmin(admin);

navigate("/admin/dashboard");
```

O login é feito através do backend.

`AdminConvidados.tsx`

Responsável pelo gerenciamento das famílias.

Permite:

- listar famílias;
- pesquisar famílias;
- criar;
- editar;
- excluir;
- visualizar detalhes.

A busca é feita através de:

```
const dados = await listarFamilias();
setFamilias(dados);
```

`AdminPresentes.tsx`

Gerencia os presentes.

Permite:

- cadastrar;
- editar;
- excluir;
- visualizar;
- acompanhar reservas.

AdminMensagens.tsx

Gerencia as mensagens enviadas pelos convidados.

AdminGaleria.tsx

Gerencia as fotos da galeria.

As fotos são enviadas para o backend, que posteriormente utiliza o Cloudinary.

---

9. admin/_components

São componentes específicos do painel.

Exemplos:

```
CardDashboard.tsx
LinhaFamilia.tsx
LinhaPresente.tsx
MenuAdmin.tsx
TabelaFamilias.tsx
TabelaMensagens.tsx
TabelaPresentes.tsx
```

CardDashboard.tsx

Mostra informações estatísticas do evento.

TabelaFamilias.tsx

Tabela com as famílias cadastradas.

LinhaFamilia.tsx

Representa uma família dentro da tabela.

Mostra informações como:

- família;
- senha;
- quantidade de convidados;
- confirmações;
- ações.

TabelaPresentes.tsx

Lista os presentes no painel.

LinhaPresente.tsx

Representa um presente individual.

TabelaMensagens.tsx

Lista as mensagens recebidas.

`MenuAdmin.tsx`

Menu de navegação do painel.

---

10. `admin/modals`

Aqui ficam as janelas modais usadas pelo administrador.

Exemplos:

```
ModalFamilia.tsx
ModalPresentes.tsx
ModalMensagem.tsx

ModalExcluirFamilia.tsx
ModalExcluirFoto.tsx
ModalExcluirMensagem.tsx
ModalExcluirPresentes.tsx
```

`ModalFamilia.tsx`

Usado para:

- cadastrar família;
- editar família;
- adicionar convidados;
- alterar senha.

A família possui:

```
Nome
Senha
Convidados
```

`ModalPresentes.tsx`

Cadastro/edição dos presentes.

`ModalMensagem.tsx`

Visualização de mensagem.

Modais de exclusão

Confirmam antes de excluir:

```
família
foto
mensagem
presente
```

---

11. `src/services`

Essa é uma das partes mais importantes da arquitetura.

Os arquivos de `services` são responsáveis por fazer a comunicação do frontend com o backend.

Por exemplo:

```
familia.service.ts
presente.service.ts
admin.service.ts
mensagem.service.ts
foto.service.ts
```

A ideia é evitar colocar chamadas `fetch()` espalhadas pelas páginas.

---

12. Configuração da API

O frontend utiliza:

```
export const API =
  import.meta.env.VITE_API;
```

Isso significa que a URL da API não fica fixa diretamente no código.

No ambiente de produção foi utilizado:

```
VITE_API=https://site-confirma-backend.onrender.com
```

Assim o frontend sabe onde está o backend.

---

13. Serviço de famílias

O `familia.service.ts` possui funções como:

```
listarFamilias()
criarFamilia()
editarFamilia()
atualizarFamilia()
excluirFamilia()
loginFamilia()
buscarFamilia()
salvarConfirmacao()
```

Por exemplo:

```
export async function loginFamilia(nome: string, senha: string) {
  const response = await fetch(`${API}/familias/login`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
```

```
    body: JSON.stringify({ nome, senha }),
  });

  if (!response.ok) {
    throw new Error("Nome ou senha inválidos.");
  }

  return response.json();
}
```

---

14. Login das famílias

O fluxo ficou:

```
Família
   ↓
Nome + senha
   ↓
Frontend
   ↓
POST /familias/login
   ↓
Backend
   ↓
Prisma
   ↓
PostgreSQL/Neon
   ↓
Família encontrada
   ↓
Frontend recebe os dados
```

O login compara:

```
nome
senha
```

com os dados existentes no banco.

Durante os testes, foi encontrado um problema importante: algumas famílias tinham **espaços extras no nome armazenado no banco**.

Depois da correção dos nomes no banco, o login passou a funcionar corretamente.

---

15. Cadastro das famílias

O administrador cadastra:

```
Nome da família
Senha
Quantidade de convidados
Nome dos convidados
```

O backend cria a família e seus convidados.

A estrutura é:

```
Familia
   │
   ├── Convidado
   ├── Convidado
   └── Convidado
```

---

16. Confirmação de presença

Quando a família acessa o site:

```
Login
 ↓
Família encontrada
 ↓
Convidados apresentados
 ↓
Convidado confirma ou não
 ↓
Frontend envia confirmação
 ↓
Backend salva
 ↓
Banco atualizado
```

A rota utilizada é:

```
PATCH /familias/confirmacao
```

O backend salva:

```
confirmado: true/false
respondidoEm: new Date()
```

---

17. Presentes

Os presentes possuem informações como:

```
id
nome
preço
imagem
reservado
reservador
```

O fluxo de reserva é:

```
Convidado
 ↓
Escolhe presente
 ↓
Modal pede nome
 ↓
Confirma reserva
 ↓
Frontend
 ↓
Backend
 ↓
```

```
Banco
 ↓
reservado = true
reservador = nome
```

Depois o botão passa a indicar que o presente foi reservado.

---

18. Mensagens

Os convidados também podem enviar mensagens.

Fluxo:

```
Convidado
 ↓
Escreve mensagem
 ↓
Frontend
 ↓
POST /mensagens
 ↓
Backend
 ↓
Prisma
 ↓
Neon
```

O administrador visualiza essas mensagens no painel.

---

19. Galeria de fotos

A galeria utiliza **Cloudinary** para armazenamento das imagens.

O fluxo é:

```
Administrador
 ↓
Seleciona foto
 ↓
Frontend
 ↓
Backend
 ↓
Upload da imagem
 ↓
Cloudinary
 ↓
Cloudinary devolve URL
 ↓
Backend salva URL no banco
```

Foi utilizado no backend o processo de upload através da função:

```
uploadImagemBuffer()
```

A URL retornada pelo Cloudinary fica armazenada no campo:

```
imagem
```

do registro correspondente no banco.

Assim, o banco **não precisa armazenar o arquivo da foto**.

Ele guarda a referência/URL da imagem.

---

20. Cloudinary

O Cloudinary foi usado especificamente para as imagens.

As configurações ficam através de variáveis de ambiente, como:

```
CLOUDINARY_CLOUD_NAME
CLOUDINARY_API_KEY
CLOUDINARY_API_SECRET
```

Essas informações são **secretas** e não devem ser colocadas no GitHub.

---

21. Backend

O backend foi criado com Express + TypeScript.

A responsabilidade dele é:

- receber requisições;
- validar informações;
- executar regras;
- acessar o banco;
- fazer upload das fotos;
- devolver respostas ao frontend.

---

22. Estrutura do BACKEND

A organização principal ficou:

```
BACKEND
│
├── src
│   │
│   ├── config
│   ├── controllers
│   ├── repositories
│   ├── routes
│   ├── services
│   ├── utils
```

```
│   └── ...
│
├── prisma
│
├── uploads
│
├── package.json
└── ...
```

---

23. `src/config`

Configurações do backend.

Existe a configuração do Prisma:

`config/prisma.ts`

Ela é responsável pela conexão do backend com o Prisma.

---

24. `src/routes`

Define as URLs da API.

Existem rotas para:

```
familias
convidados
presentes
mensagens
fotos
admins
dashboard
```

No `server.ts`:

```
app.use("/familias", familiaRoutes);
app.use("/convidados", convidadoRoutes);
app.use("/presentes", presenteRoutes);
app.use("/mensagens", mensagemRoutes);
app.use("/fotos", fotoRoutes);

app.use("/admins", adminRoutes);
app.use("/dashboard", dashboardRoutes);
```

---

25. Principais endpoints

Famílias

```
POST   /familias
GET    /familias
GET    /familias/:id
PUT    /familias/:id
DELETE /familias/:id
```

```
POST   /familias/login
PATCH  /familias/confirmacao
```

Administradores

```
POST   /admins
POST   /admins/login
PATCH  /admins/redefinir-senha
```

Presentes

Possuem endpoints para:

```
listar
criar
editar
excluir
reservar/atualizar
```

Mensagens

Endpoints para:

```
criar
listar
excluir
```

Fotos

Endpoints para:

```
upload
listar
excluir
```

---

26. `src/controllers`

Os controllers recebem a requisição HTTP.

Exemplo:

```
Request
   ↓
Controller
   ↓
Service
```

O controller não deveria concentrar toda a regra do sistema.

Ele recebe os dados e chama o service.

---

27. `src/services`

Aqui ficam as regras de negócio.

Por exemplo:

```
AdminService
FamiliaService
PresenteService
MensagemService
...
```

O `AdminService` possui:

```
criar()
login()
redefinirSenha()
```

Para criação do administrador:

```
const senhaCriptografada =
  await bcrypt.hash(senha, 10);
```

Isso faz com que a senha não seja armazenada em texto puro.

---

28. `src/repositories`

Os repositories fazem o acesso ao banco através do Prisma.

Exemplo:

```
prisma.admin.findUnique(...)
```

ou:

```
prisma.familia.findMany(...)
```

A arquitetura ficou:

```
ROUTE
 ↓
CONTROLLER
 ↓
SERVICE
 ↓
REPOSITORY
 ↓
PRISMA
 ↓
NEON
```

Essa separação facilita manutenção e reutilização.

---

29. Administrador

O administrador possui:

```
id
nome
email
senha
```

A senha é protegida usando:

```
bcrypt
```

No login:

```
const senhaCorreta =
  await bcrypt.compare(
    senha,
    admin.senha
  );
```

Se estiver correta, o backend retorna:

```
{
  id: admin.id,
  nome: admin.nome,
  email: admin.email
}
```

---

30. Cadastro do administrador

O backend possui a rota:

```
POST /admins
```

No Render:

```
POST https://site-confirma-backend.onrender.com/admins
```

O corpo utilizado é:

```
{
  "nome": "Nome do administrador",
  "email": "email@cliente.com",
  "senha": "SenhaDoCliente123"
}
```

O backend transforma a senha em hash com bcrypt antes de salvar.

Isso permite que você crie o administrador de cada novo cliente sem precisar modificar o código.

PARA TROCAR O ADMINISTRADOR SEGUIR ESSES PASSOS:

1. Abra o Postman

Clique em **New → HTTP Request**.

2. Escolha o método

À esquerda da URL, selecione:

```
POST
```

3. Coloque a URL

Use exatamente:

```
https://site-confirma-backend.onrender.com/admins
```

4. Vá em Body

Clique em:

Body → raw → JSON

Vai aparecer uma área para colocar o conteúdo.

5. Coloque os dados do cliente

Por exemplo:

```
{
  "nome": "Maria",
  "email": "maria@email.com",
  "senha": "Maria123456"
}
```

⚠ **Use os dados reais que você quer entregar ao cliente.** A senha que você colocar aqui será a senha que ele usará para entrar.

6. Clique em Send

Se estiver tudo certo, você deverá receber uma resposta parecida com:

```
{
  "id": 1,
  "nome": "Maria",
  "email": "maria@email.com"
}
```

A senha **não deve aparecer na resposta**, o que é bom.

7. Depois teste no site

Abra o endereço do seu Vercel e vá para:

Acesso Privado

Use:

```
E-mail: maria@email.com
Senha: Maria123456
```

Se entrar no painel, está funcionando. 🥕

---

31. Dashboard

O dashboard apresenta informações estatísticas do evento.

Por exemplo:

- quantidade de famílias;
- quantidade de convidados;
- confirmações;
- presentes;
- mensagens;
- outros dados utilizados no projeto.

A rota é:

```
/dashboard
```

---

32. Banco de dados — Neon

Foi utilizado o **Neon** como banco de dados PostgreSQL.

O banco contém as informações do site.

Entre elas:

```
Admin
Familia
Convidado
Presente
Mensagem
Foto
```

A conexão é feita através da variável:

```
DATABASE_URL
```

A senha/conexão do banco **não deve ser publicada no GitHub**.

---

33. Prisma

O Prisma foi utilizado como ORM.

Ele permite fazer operações como:

```
prisma.familia.findMany()
prisma.familia.create()
prisma.presente.update()
prisma.admin.findUnique()
```

Em vez de escrever SQL manualmente para cada operação.

---

34. GitHub

Depois do desenvolvimento, o projeto foi colocado no GitHub.

Repositório:

AndreiaBernardo/Site-confirma

Estrutura:

```
Site-confirma
├── BACKEND
└── FRONTEND
```

O GitHub funciona como:

- armazenamento do código;
- histórico das alterações;
- origem dos deployments;
- controle de versões.

---

35. `.gitignore`

Arquivos sensíveis não devem ser enviados para o GitHub.

Entre eles:

```
.env
```

Além de:

```
node_modules
```

e outros arquivos gerados.

As informações secretas ficam nas configurações dos serviços de deploy.

---

36. Deploy do backend — Render

O backend foi publicado no **Render**.

O projeto utilizado no Render aponta para:

```
BACKEND
```

O endereço público ficou:

Backend — Render

Esse endereço é usado pelo frontend através de:

```
VITE_API
```

---

37. Deploy do frontend — Vercel

O frontend foi publicado na **Vercel**.

O Vercel fica responsável por hospedar a parte visual do site.

O frontend utiliza a variável:

```
VITE_API
```

apontando para:

```
https://site-confirma-backend.onrender.com
```

O fluxo de produção ficou:

```
CONVIDADO
   ↓
VERCEL
   ↓
FRONTEND
   ↓
RENDER
   ↓
BACKEND
   ↓
PRISMA
   ↓
NEON
```

E para fotos:

```
ADMINISTRADOR
   ↓
VERCEL
   ↓
BACKEND / RENDER
   ↓
CLOUDINARY
```

---

38. Processo de atualização do site

Depois que o site ficou pronto, você passou a utilizar este processo:

Frontend

Dentro da pasta:

```
FRONTEND
```

testa com:

```
npm run dev
```

Depois faz:

```
npm run build
```

Backend

Dentro de:

```
BACKEND
```

testa/faz build com:

```
npm run build
```

Se os dois estiverem OK, volta para a pasta principal:

```
CONFIRMAÇÃO
```

e executa:

```
git status
```

Depois:

```
git add .
```

Depois:

```
git commit -m "Personalização para novo cliente"
```

E:

```
git push origin main
```

---

39. O que acontece depois do `git push`

O GitHub recebe as alterações.

A Vercel detecta o novo commit e inicia automaticamente um novo deploy do frontend.

Como o backend não foi alterado, não é necessário fazer novo deploy do Render nessa situação.

Fluxo:

```
VS CODE
```

```
   ↓
git add .
   ↓
git commit
   ↓
git push
   ↓
GITHUB
   ↓
VERCEL
   ↓
NOVO DEPLOY
```

---

40. Primeiro site — Yasmim

A primeira versão foi criada para:

Yasmim — 15 anos

Com identidade visual baseada em:

- lilás;
- roxo;
- prata;
- borboletas;
- elementos de festa de 15 anos.

Foram cadastrados:

- famílias;
- convidados;
- presentes;
- mensagens;
- fotos.

A confirmação de presença funcionou.

A reserva de presentes funcionou.

O painel administrativo funcionou.

A galeria funcionou.

O sistema foi testado em computador e celular.

---

41. Problemas encontrados e resolvidos

Durante o desenvolvimento, alguns problemas importantes foram encontrados.

Login das famílias

Alguns nomes tinham espaços extras no banco.

Isso causava:

```
Nome ou senha inválidos
```

A correção foi ajustar os nomes armazenados no banco.

Presente aparecendo duas vezes

Foi identificado que:

```
addTransaction(data)
```

estava sendo chamado duas vezes em outro contexto do projeto financeiro anterior; no site de confirmação, o fluxo de presentes foi posteriormente ajustado e passou a funcionar corretamente.

Responsividade

As ações do painel administrativo não apareciam corretamente em alguns celulares.

Foram feitas alterações nas tabelas/componentes para melhorar a visualização dos botões de:

- editar;
- excluir;
- visualizar.

Depois dos ajustes, o funcionamento ficou adequado no celular.

Arquivos vazios

Foram removidos:

```
Footer.tsx
Modal.tsx
```

porque você confirmou que estavam vazios/não eram utilizados.

---

42. Modelo para novos clientes

Agora a ideia é usar esse projeto como **modelo/base**.

Você não precisa reconstruir tudo novamente.

A estrutura permanece:

```
FRONTEND
BACKEND
PRISMA
```

```
NEON
CLOUDINARY
RENDER
VERCEL
```

Para um novo cliente, normalmente serão alterados:

```
Nome da aniversariante
Cores
Fotos
Textos
Data
Horário
Local
Famílias
Convidados
Presentes
Informações do evento
```

A estrutura técnica permanece a mesma.

---

43. Fluxo completo do sistema

Convidado

```
LINK DO SITE
     ↓
VERCEL
     ↓
PÁGINA INICIAL
     ↓
INFORMAÇÕES DO EVENTO
     ↓
CONFIRMAÇÃO / PRESENTES / MENSAGENS
     ↓
BACKEND
     ↓
NEON
```

Família

```
Nome + senha
     ↓
FRONTEND
     ↓
POST /familias/login
     ↓
BACKEND
     ↓
PRISMA
     ↓
NEON
     ↓
Família + convidados
     ↓
Confirmação
```

Administrador

```
E-mail + senha
     ↓
FRONTEND
     ↓
POST /admins/login
     ↓
BACKEND
     ↓
bcrypt
     ↓
PRISMA
     ↓
NEON
     ↓
Painel administrativo
```

Fotos

```
ADMIN
 ↓
FRONTEND
 ↓
BACKEND
 ↓
CLOUDINARY
 ↓
URL DA FOTO
 ↓
NEON
```

Presentes

```
Convidado
 ↓
Escolhe presente
 ↓
Nome
 ↓
Reserva
 ↓
BACKEND
 ↓
NEON
 ↓
Presente reservado
```

---

44. Resumo dos serviços usados

Serviço	Uso no projeto
React	Interface do site
TypeScript	Tipagem do código
Vite	Desenvolvimento/build do frontend
Tailwind CSS	Estilização
React Router	Navegação entre páginas
Lucide React	Ícones
Node.js	Execução do backend
Express	API/backend

Serviço	Uso no projeto
Prisma	ORM e acesso ao banco
PostgreSQL	Banco de dados
Neon	Hospedagem do PostgreSQL
bcrypt	Criptografia/hash das senhas
Cloudinary	Armazenamento das fotos
Git	Controle de versão
GitHub	Repositório do código
Render	Hospedagem do backend
Vercel	Hospedagem do frontend
Postman	Testes das rotas da API

---

45. Endereços importantes

GitHub

Repositório Site-confirma

Backend

API no Render

Banco

Neon — PostgreSQL

A conexão é feita pela variável:

`DATABASE_URL`

Fotos

Cloudinary

As imagens ficam armazenadas lá e o banco guarda suas URLs.

Frontend

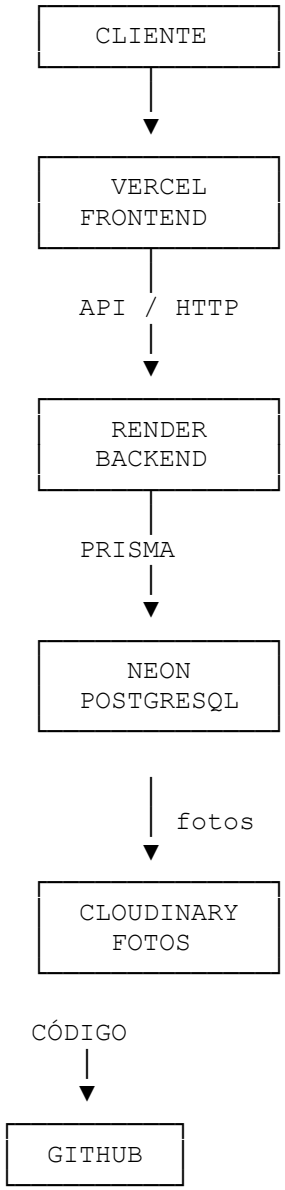
Vercel

O link específico do site pode mudar conforme o projeto/deployment, então é o endereço que aparecer no painel do Vercel.

---

46. O mais importante para você guardar

A arquitetura final do projeto é:



Essa é a base que você agora pode reutilizar para outros clientes. Você personaliza o conteúdo e a identidade visual, enquanto mantém a arquitetura `FRONTEND + BACKEND + Neon + Cloudinary + Render + Vercel`.